Rails Internationalization (I18n) API

The Ruby I18n (shorthand for *internationalization*) gem which is shipped with Ruby on Rails (starting from Rails 2.2) provides an easy-to-use and extensible framework for **translating your application to a single custom language**other than English or for **providing multi-language support** in your application.
The process of "internationalization" usually means to abstract all strings and other locale specific bits (such as date or currency formats) out of your application. The process of "localization" means to provide translations and localized formats for these bits.[1]

The Ruby I18n framework provides you with all necessary means for internationalization/localization of your Rails application. You may, also use various gems available to add additional functionality or features. See the rails-i18n gem for more information.

1 How I18n in Ruby on Rails Works

Internationalization is a complex problem. Natural languages differ in so many ways (e.g. in pluralization rules) that it is hard to provide tools for solving all problems at once. For that reason the Rails I18n API focuses on:

- providing support for English and similar languages out of the box
- making it easy to customize and extend everything for other languages

As part of this solution, **every static string in the Rails framework** - e.g. Active Record validation messages, time and date formats - **has been internationalized**, so *localization* of a Rails application means "over-riding" these defaults.

1.1 The Overall Architecture of the Library

Thus, the Ruby I18n gem is split into two parts:

- The public API of the i18n framework - a Ruby module with public methods that define how the library works
- A default backend (which is intentionally named *Simple* backend) that implements these methods

As a user you should always only access the public methods on the I18n module, but it is useful to know about the capabilities of the backend.

It is possible (or even desirable) to swap the shipped Simple backend with a more powerful one, which would store translation data in a relational database, GetText dictionary, or similar. See section Using different backends below.

1.2 The Public I18n API

The most important methods of the I18n API are:

```
translate # Lookup text translations
```